Machine Coding: Sortable Product Table

- **Problem:** Show a product list (name, category, price, rating, stock) in a table where clicking a column header sorts by that column: first click ascending, second descending, third back to the original order. Strings sort naturally and case-insensitively, numbers numerically, booleans meaningfully. Missing values go last, ties stay stable, the original data is never mutated, and the sort is accessible to keyboard and screen-reader users.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Sort cycle, types, null rule decided
Use cases + architecture + state	7 min	Table, `{ key, direction }` state
Render the table + one-column price sort	10 min	Working sort
Generic comparator (numbers, strings, booleans, empties, ties)	13 min	Correct on all columns
Header cycle, `aria-sort`, reset, live label	10 min	Accessible UI
Self-check + manual test + wrap-up	15 min	Proven sort + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"I store only `{ key, direction }`. The sorted list is derived from the original data every render, so I never mutate the source and 'reset' is just clearing the key."* Then mention that `Array.prototype.sort` mutates and that it's stable since ES2019.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Table with columns Product, Category, Price, Rating, Stock.
- Click a header: ascending → descending → original order (3-state cycle). Clicking another column starts it at ascending.
- Visible indicator (▲ / ▼ / ↕) and a text label "Sorted by Price, ascending".
- Reset button back to the original order.
- Correct comparisons: numbers numerically (not as text: "1000" vs "999"), strings case-insensitive and number-aware ("Item 2" before "Item 10"), booleans (in stock first when descending).
- Missing values (`null` / `undefined` /"") always at the bottom; equal values keep a stable order.

Non-functional requirements

Concern	What it means here
Performance	O(n log n) per sort; memoised on `sort` so re-renders that don't change the sort don't re-sort. Fine up to tens of thousands of rows.
Scalability	Column config array: a new sortable column is one entry. Server-side sort later uses the same `{ key, direction }`.
Error handling	Mixed or missing values never throw and never produce `NaN` comparisons; empty data shows a message.
Accessibility	Real `<button>` in each `<th>`, `aria-sort` on the header, `scope="col"`, `<caption>`, live region announcing the sort, visible focus ring.
Reusability	`sortItems(items, { key, direction })`, `compareValues` and `nextSort` are pure and work for any list.
Extensibility	Multi-column sort, URL sync or server-side sorting plug in without changing the components.

Clarifying questions to ask the interviewer

1. Which columns are sortable, and is it table headers or a "Sort by" dropdown (or both)?
2. Should the third click return to the original order, or only toggle asc/desc?
3. How should strings sort: case-insensitive? Natural numbers ("Item 2" < "Item 10")? Which locale?
4. Where do missing values go: always last, or by direction?
5. Should sorting combine with filters / pagination / the URL?
6. How big is the data: client-side sort, or must the server sort?

Assumptions we will build with

- Sortable table headers for all 5 columns; 3-state cycle; a Reset button.
- Strings: `Intl.Collator` with `numeric: true, sensitivity: "base"`.
- Missing values always last, in both directions.

- Ties broken by `id`, so the order is deterministic.
- Local data (`data/product.js`, 10 items), client-side sort; no URL sync (an extension).

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Original order, "Original order", all icons ↕, `aria-sort="none"`, Reset disabled
Click "Price"	Low → high; ▲ on Price; "Sorted by Price, ascending"; `aria-sort="ascending"`
Click "Price" again	High → low; ▼; "descending"
Click "Price" a third time	Original order restored; ↕
Price sorted, click "Product"	Product A → Z (new column always starts ascending)
Sort by Product	Case-insensitive, natural: "apple", "Banana", "Item 2", "Item 10"
Sort by Stock descending	In-stock rows first
Two products with the same price	Keep a stable order (by id)
A product with no rating	Always at the bottom, ascending or descending
Click Reset	Original order; Reset disabled again
Keyboard: Tab to a header, press Enter/Space	Same as click; focus ring visible
Screen reader	Hears the header's sort state and the live label after each change
Phone width	The table scrolls horizontally inside its box; the page doesn't
No products	"No products to show."

3. Design the architecture

Module	Responsibility
`App`	Owns `sort`; derives the sorted list and label; renders the toolbar and table
`utils/sort.js`	`compareValues`, `sortItems`, `nextSort`: pure sorting logic
`SortableHeader`	One `<th>` with a button, the icon and `aria-sort`
`data/product.js`	The product list (source of truth, never mutated)

```
App  (sort = { key, direction })
├── toolbar: live label + Reset
└── <table>
     ├── <thead> COLUMNS.map → SortableHeader (column, sort, onSort)
     └── <tbody> sortedProducts.map → <tr key={id}>
utils/sort.js  ← nextSort (click cycle), sortItems (copy + comparator), compareValues (types)
```

Start simple. The first version is the existing price dropdown: `[...products].sort((a, b) => a.price - b.price)`. Generalise it to any key, then add the header cycle, then the edge cases.

4. Design the state / data model

State	Purpose
`sort.key`	Column being sorted (`null` = original order)
`sort.direction`	`"asc" | "desc" | "none"`

```
const [sort, setSort] = useState({ key: null, direction: "none" }); // the ONLY state

const sortedProducts = useMemo(() => sortItems(products, sort), [sort]); // derived
const sortLabel = sort.key ? `Sorted by ${label}, ${DIRECTION_TEXT[sort.direction]}` : "Original order";

const onSort = (key) => setSort((s) => nextSort(s, key)); // functional update: no stale state
```

Senior note

- **Derived, not stored:** the first version stored *both* `sortedOrder` and `sortedItem`, two states that can disagree (and a second copy of the data). Store only the sort *instruction*; the sorted list, header icons, `aria-sort` and the label are all derived.
- **Original order is free:** because we never mutate `products`, "no sort" is simply returning the original array.
- **Ownership:** the data module owns the products; `App` owns how to view them; `SortableHeader` only reports clicks.

5. Design the data structures and contracts

Product (stable numeric `id`, used as the React key and the tie-breaker):

```
{ "id": 1, "name": "Wireless Mouse", "category": "Electronics", "price": 499, "rating": 4.3, "inStock": true }
```

Column config (drives the headers; a new column is one entry):

```
{ key: "price", label: "Price", align: "right" }
```

Contracts

```
compareValues(a, b)              → negative | 0 | positive    // numbers, booleans, strings (Intl.Collator)
sortItems(items, { key, direction }, tieBreaker = "id") → NEW array (or `items` itself when direction "none")
```

```
nextSort({ key, direction }, clickedKey) → next { key, direction }   // asc → desc → none
<SortableHeader column={col} sort={sort} onSort={(key) => …} />
```

Server-side version: `GET /products?sort=price&order=desc` with the same `{ key, direction }`.

6. Trace the data flow

User clicks "Price" twice:

```
Click 1 on the Price header button
  ↓ onSort("price") → setSort(s => nextSort(s, "price")) → { key: "price", direction: "asc" }
  ↓ React re-renders App
  ↓ useMemo: sort changed → sortItems(products, sort)
  ↓   [...products].sort((x, y) => empties last || 1 * (x.price – y.price) || x.id – y.id)
  ↓ UI: rows 299 … 2999, ▲ on Price, aria-sort="ascending", live label announced
Click 2 → { key: "price", direction: "desc" } → same flow with sign = –1 → rows 2999 … 299, ▼
Click 3 → { key: null, direction: "none" } → sortItems returns the original array → original order
```

7. Important technical challenges

Challenge 1: never mutate the source (and don't store a sorted copy)

- *Problem:* `products.sort(...)` sorts **in place**: it changes the original array, so the "original order" is lost forever, and in React state it keeps the same reference, so nothing re-renders. Storing `sortedItem` as state duplicates the data.
- *Solution:* Copy before sorting and derive the list from the sort state.

```
if (!key || direction === "none") return items;          // original order, same reference
return [...items].sort(comparator);                       // or items.toSorted(comparator) in ES2023
```

Challenge 2: one comparator for every type

- *Problem:* `a.price - b.price` only works for numbers. For strings, `"b" - "a"` is `NaN`, and the default `sort()` compares as text, so `[1000, 999].sort()` stays `[1000, 999]`, and "Zebra" sorts before "apple" (uppercase first). Booleans need a meaning.
- *Solution:* Pick the comparison by type; use `Intl.Collator` for human-friendly strings.

```
const collator = new Intl.Collator(undefined, { numeric: true, sensitivity: "base" });
export function compareValues(a, b) {
  if (typeof a === "number" && typeof b === "number") return a - b;
  if (typeof a === "boolean" && typeof b === "boolean") return Number(a) - Number(b);
  return collator.compare(String(a), String(b));
}
```

Challenge 3: missing values and ties

- *Problem:* A product with `rating: null`: `null - 4.5` is `-4.5`, so it jumps to the top in ascending order and to the bottom in descending order (it should stay at the bottom in both). Two products at ₹899 can swap places between renders if the comparator isn't consistent.
- *Solution:* Handle empties *before* applying the direction, and break ties with a stable key.

```
if (isEmpty(a) || isEmpty(b)) return isEmpty(a) - isEmpty(b);        // empties last, both directions
return sign * compareValues(a, b) || compareValues(x[tieBreaker], y[tieBreaker]);
```

Challenge 4: an accessible 3-state header

- *Problem:* A clickable `<th onClick>` can't be focused with the keyboard, and screen readers don't know the column is sorted. The cycle logic spread across handlers gets buggy.
- *Solution:* A `<button>` inside each `<th>`, `aria-sort` on the `<th>`, a live text label, and the cycle as a pure function.

```
export function nextSort(current, key) {
  if (current.key !== key) return { key, direction: "asc" };
  if (current.direction === "asc") return { key, direction: "desc" };
  return { key: null, direction: "none" };
}
<th scope="col" aria-sort={ARIA_SORT[direction] ?? "none"}><button onClick={() => onSort(column.key)}>…
</button></th>
```

8. Implement in the correct order

#	Step	Working result
1	Render the table from `data/product.js` with `key={p.id}`	Data visible
2	Price sort with `[...products].sort((a, b) => a.price - b.price)`	One working sort
3	`sort = { key, direction }` state; derive the list	Single source of truth
4	Generalise to any key + direction sign	All columns sortable
5	`compareValues` by type (numbers, booleans, `Intl.Collator`)	Correct ordering
6	Empty values last + `id` tie-breaker	Deterministic results
7	`nextSort` 3-state cycle; header buttons	Click-to-sort UI
8	Icons, `aria-sort`, live label, Reset	Clear + accessible
9	`useMemo` on `sort`	No re-sort on unrelated renders
10	Node self-check (types, empties, ties, no mutation, cycle)	Logic proven
11	Responsive table wrapper, focus ring, empty state	Polished

9. Cover all states and edge cases

State / edge case	Handling
Initial	Original order, ↕ icons, Reset disabled
Sorted ascending / descending	▲ / ▼, `aria-sort`, live label
Back to original	Third click or Reset; the original array is returned
Empty data	"No products to show."
One item	Sorting is a no-op; no errors
Equal values	Tie broken by `id` → stable, deterministic
`null` / `undefined` / "" / `NaN`	Always last, both directions; shown as "—"
Numbers stored as numbers	Numeric compare (999 < 1000)
Mixed-case strings	Case-insensitive ("apple" next to "Apple")
Numbers inside strings	Natural order ("Item 2" < "Item 10")
Accented strings	Collator handles locale order ("é" near "e")
Booleans	`false < true`; descending puts in-stock first
Switch column	The new column starts ascending
Keyboard only	Buttons focusable, Enter/Space work, visible focus
Narrow screen	Horizontal scroll inside the table box

10. Final structure

```
_07_sorting_features/
├── index.html
├── data/
│   └── product.js             ← source data (never mutated)
└── src/
    ├── main.jsx
    ├── index.css              ← global reset
    ├── App.jsx                ← sort state, derived list + label, table
    ├── App.css
    ├── utils/
    │   └── sort.js            ← compareValues, sortItems, nextSort
    └── components/
        └── SortableHeader.jsx ← accessible sortable <th>
```

11. Full working code

`src/utils/sort.js`

```
// Case-insensitive, number-aware string compare: "Item 2" < "Item 10", "apple" == "Apple".
const collator = new Intl.Collator(undefined, { numeric: true, sensitivity: "base" });

const isEmpty = (v) => v === null || v === undefined || v === "" || Number.isNaN(v);

// Ascending compare for one value type. Empty values are handled by the caller.
export function compareValues(a, b) {
  if (typeof a === "number" && typeof b === "number") return a - b;
  if (typeof a === "boolean" && typeof b === "boolean") return Number(a) - Number(b); // false < true
  return collator.compare(String(a), String(b));
}

// Next sort state when a column header is clicked: asc -> desc -> none (original order).
export function nextSort(current, key) {
  if (current.key !== key) return { key, direction: "asc" };
  if (current.direction === "asc") return { key, direction: "desc" };
  return { key: null, direction: "none" };
}

// Returns a NEW sorted array (never mutates `items`). Empty values always go last,
// ties keep a stable order via the `tieBreaker` key.
export function sortItems(items, { key, direction }, tieBreaker = "id") {
  if (!key || direction === "none") return items;
  const sign = direction === "desc" ? -1 : 1;

  return [...items].sort((x, y) => {
    const a = x[key];
    const b = y[key];
    if (isEmpty(a) || isEmpty(b)) return isEmpty(a) - isEmpty(b); // empties last in both directions
    return sign * compareValues(a, b) || compareValues(x[tieBreaker], y[tieBreaker]);
  });
}
```

src/App.jsx

```
import { useMemo, useState } from "react";
import "./App.css";
import products from "../data/product.js";
import { nextSort, sortItems } from "./utils/sort";
import SortableHeader from "./components/SortableHeader";

const COLUMNS = [
  { key: "name", label: "Product" },
  { key: "category", label: "Category" },
  { key: "price", label: "Price", align: "right" },
  { key: "rating", label: "Rating", align: "right" },
```

```
  { key: "inStock", label: "Stock" },
];
const DIRECTION_TEXT = { asc: "ascending", desc: "descending" };
const priceFormat = new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR",
maximumFractionDigits: 0 });

function App() {
  const [sort, setSort] = useState({ key: null, direction: "none" }); // the ONLY state: never store the
sorted list

  // Derived. useMemo only because sorting is O(n log n) and the table can grow; 10 rows wouldn't need
it.
  const sortedProducts = useMemo(() => sortItems(products, sort), [sort]);

  const sortLabel = sort.key
    ? `Sorted by ${COLUMNS.find((c) => c.key === sort.key).label}, ${DIRECTION_TEXT[sort.direction]}`
    : "Original order";

  return (
    <main className="container">
      <h1>Sorting Products</h1>

      <div className="toolbar">
        <p aria-live="polite">{sortLabel}</p>
        <button type="button" onClick={() => setSort({ key: null, direction: "none" })} disabled=
{!sort.key}>
          Reset
        </button>
      </div>

      {sortedProducts.length === 0 ? (
        <p className="empty">No products to show.</p>
      ) : (
        <div className="table-wrap">
          <table>
            <caption className="sr-only">Products. Click a column header to sort.</caption>
            <thead>
              <tr>
                {COLUMNS.map((col) => (
                  <SortableHeader key={col.key} column={col} sort={sort} onSort={(key) => setSort((s) =>
nextSort(s, key))} />
                ))}
              </tr>
            </thead>
            <tbody>
              {sortedProducts.map((p) => (
                <tr key={p.id}>
```

```
                  <td>{p.name}</td>
                  <td>{p.category}</td>
                  <td className="num">{priceFormat.format(p.price)}</td>
                  <td className="num">{p.rating == null ? "—" : `★ ${p.rating.toFixed(1)}`}</td>
                  <td>
                    <span className={`stock ${p.inStock ? "in" : "out"}`}>{p.inStock ? "In stock" : "Out
of stock"}</span>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </main>
  );
}

export default App;
```

src/components/SortableHeader.jsx

```
const ARIA_SORT = { asc: "ascending", desc: "descending" };
const ICON = { asc: "▲", desc: "▼" };

// A <th> with a real <button>, so sorting works with mouse, keyboard and screen readers.
const SortableHeader = ({ column, sort, onSort }) => {
  const direction = sort.key === column.key ? sort.direction : "none";

  return (
    <th scope="col" aria-sort={ARIA_SORT[direction] ?? "none"} className={column.align === "right" ?
"num" : ""}>
      <button type="button" className="sort-btn" onClick={() => onSort(column.key)}>
        {column.label}
        <span className={`sort-icon ${direction !== "none" ? "active" : ""}`} aria-hidden="true">
          {ICON[direction] ?? "↕"}
        </span>
      </button>
    </th>
  );
};

export default SortableHeader;
```

src/App.css

```
body { background: #f3f4f6; font-family: system-ui, sans-serif; color: #111827; }
.container { max-width: 860px; margin: 32px auto; padding: 20px; background: white; border-radius: 10px;
}
h1 { text-align: center; margin-bottom: 16px; font-size: 24px; }

.toolbar { display: flex; justify-content: space-between; align-items: center; margin-bottom: 12px;
color: #4b5563; }
.toolbar button { padding: 6px 14px; border: none; border-radius: 6px; background: #4f46e5; color:
white; cursor: pointer; }
.toolbar button:disabled { opacity: 0.4; cursor: not-allowed; }

.table-wrap { overflow-x: auto; border: 1px solid #e5e7eb; border-radius: 8px; }
table { width: 100%; border-collapse: collapse; font-size: 14px; }
th, td { padding: 10px 12px; text-align: left; border-bottom: 1px solid #f3f4f6; white-space: nowrap; }
th { background: #f9fafb; padding: 0; }
.num { text-align: right; }
tbody tr:hover { background: #f9fafb; }

.sort-btn { width: 100%; display: flex; align-items: center; gap: 6px; padding: 10px 12px; border: none;
background: none; font: inherit; font-weight: 600; cursor: pointer; color: inherit; }
th.num .sort-btn { justify-content: flex-end; }
.sort-btn:hover { background: #eef2ff; }
.sort-btn:focus-visible { outline: 2px solid #4f46e5; outline-offset: -2px; }
.sort-icon { font-size: 11px; color: #9ca3af; }
.sort-icon.active { color: #4f46e5; }

.stock { font-weight: 600; font-size: 13px; }
.stock.in { color: #15803d; }
.stock.out { color: #dc2626; }
.empty { text-align: center; padding: 32px; color: #6b7280; }

.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); white-
space: nowrap; }
```

src/index.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
```

```
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

`data/product.js` is the existing data file in the task folder (array of `{ id, name, category, price, rating, inStock }`, `export default products`).

12. Wrap-up

Edge cases handled: no mutation of the source, original order restorable, typed comparisons (numbers, natural case-insensitive strings, booleans), empties always last, stable ties, 3-state header cycle, keyboard and screen-reader support, empty data, narrow screens.

Trade-offs to mention to the interviewer

- *3-state vs 2-state cycle:* the third state gives the original order back without a separate button, but some users find it surprising, so ask. I also added Reset.
- *`useMemo`:* not needed for 10 rows; it helps when the list is large or the component re-renders often for other reasons.
- *Client-side vs server-side sort:* client-side is instant for loaded data; with pagination, the server must sort (otherwise you only sort the current page, which is wrong).
- *`Intl.Collator` vs `localeCompare`:* same results, but a reused collator is much faster when sorting many strings.
- *Empties always last vs following the direction:* the most common UX expectation; make it configurable if needed.
- *`[...arr].sort` vs `arr.toSorted`:* `toSorted` (ES2023) is clearer but needs modern browsers or a polyfill.

What to add with more time

- Multi-column sort (Shift+click adds a secondary key), shown as "1 ▲, 2 ▼".
- Sync the sort to the URL (`?sort=price&dir=desc`) so it survives refresh and is shareable.
- Combine with search, filters and pagination (sort → then paginate; reset to page 1 on sort change).
- Server-side sorting for large data; virtualised rows for thousands of items.
- Unit tests with Vitest for `sortItems` / `nextSort`; React Testing Library for click cycles and `aria-sort`.

13. Counter questions the interviewer may ask

Requirements and design

Q1. Does `Array.prototype.sort` mutate? Why does it matter in React?

Yes, it sorts in place and returns the same array. In React, mutating state or props means the reference doesn't change (no re-render) and the original order is lost. Copy first: `[...items].sort(fn)` or

`items.toSorted(fn)`.

Q2. Why does `[1000, 999, 25].sort()` give `[1000, 25, 999]`?

Without a comparator, `sort` converts values to strings and compares them as text, character by character ("1" < "2" < "9"). Always pass a comparator for numbers: `(a, b) => a - b`.

Q3. What must a comparator return?

A negative number if `a` comes first, positive if `b` comes first, `0` if they're equal. It must be consistent (same inputs → same result) and transitive, or the result is unpredictable.

Q4. Is JavaScript's sort stable? Why do you still add a tie-breaker?

Since ES2019, yes: equal elements keep their input order. A tie-breaker (`id`) makes the order independent of the input order. That matters when the data comes from an API in varying order, or when you sort from an already sorted list.

State design

Q5. Why store `{ key, direction }` instead of the sorted list?

The sorted list is derived data: storing it creates two sources of truth (the first version had `sortedOrder` and `sortedItem`, which can disagree). The instruction is tiny; the list is recomputed from the untouched source.

Q6. Why the functional update `setSort((s) => nextSort(s, key))`?

The next state depends on the previous one. The functional form always reads the latest state, even if several updates are batched; reading `sort` from the closure could use a stale value.

Q7. Is `useMemo` necessary here?

Not for 10 rows. It prevents re-sorting on renders where `sort` didn't change (e.g. other state in a bigger page). I'd keep it because sorting is O(n log n) and tables tend to grow, and say it's an optimisation, not for correctness.

Hard parts

Q8. How do you sort strings correctly for humans?

`Intl.Collator(undefined, { numeric: true, sensitivity: "base" })`: case-insensitive, accent-aware for the user's locale, and natural numbers ("Item 2" < "Item 10"). Create it once and reuse `collator.compare`; that's faster than calling `localeCompare` in a loop.

Q9. Where should `null` values go, and why handle them before the direction?

Usually always last. If you just multiply the comparison by −1 for descending, nulls jump from the top to the bottom. Checking empties first, *outside* the sign, keeps them last in both directions.

Q10. How would you sort dates?

Store them as ISO strings or timestamps and compare numbers: `new Date(a) - new Date(b)`, or better, pre-convert to timestamps once (parsing dates inside the comparator runs O(n log n) times).

Q11. How would you implement multi-column sort?

Keep an array of sort keys: `[{ key: "category", direction: "asc" }, { key: "price", direction: "desc" }]`. The comparator walks the list and returns the first non-zero result. In the UI, Shift+click adds or toggles a secondary key; show priority numbers on the headers.

Implementation steps

Q12. How did you make the headers accessible?

A `<button>` inside each `<th>` (focusable, Enter/Space for free), `aria-sort="ascending|descending|none"` on the `<th>`, `scope="col"`, a `<caption>`, a live region announcing "Sorted by Price, ascending", and the icon marked `aria-hidden` (the state is in the attributes).

Q13. How would you test the sorting?

Pure-function tests: each type, empties last in both directions, ties by id, a frozen input array (proves no mutation), and the asc → desc → none cycle. Then React Testing Library: click a header three times and assert the row order and `aria-sort` each time.

Q14. Write a sort without `Array.prototype.sort` (e.g. merge sort).

```
function mergeSort(arr, cmp) {
  if (arr.length <= 1) return arr;
  const mid = arr.length >> 1;
  const left = mergeSort(arr.slice(0, mid), cmp), right = mergeSort(arr.slice(mid), cmp);
  const out = [];
  let i = 0, j = 0;
  while (i < left.length && j < right.length) out.push(cmp(left[i], right[j]) <= 0 ? left[i++] :
right[j++]);
  return out.concat(left.slice(i), right.slice(j));
}
```

O(n log n), stable (`<= 0` keeps the left item first), doesn't mutate.

Performance and scale

Q15. What algorithm does V8 use, and what's the complexity?

TimSort (a hybrid of merge sort and insertion sort): O(n log n) worst case, O(n) on already sorted data, stable. Note that the comparator is called O(n log n) times, so keep it cheap.

Q16. Sorting 100,000 rows freezes the UI. What do you do?

Precompute sort keys once (e.g. lowercase strings, timestamps), reuse an `Intl.Collator`, memoise, and virtualise the rendering (the DOM is usually the bigger cost). If still slow, sort in a Web Worker or on the server.

Q17. The table is paginated on the server. Can you sort on the client?

No: you'd only sort the current page, which is wrong. Send `sort` and `order` to the API, reset to page 1 when the sort changes, and show a loading state.

Trade-offs and "why not X?"

Q18. Why not lodash `orderBy` or TanStack Table?

In production, TanStack Table gives sorting, multi-sort, filtering and pagination with accessibility hooks, and it's worth it for complex grids. In an interview, writing the comparator shows you understand `sort`, stability, types and nulls.

Q19. Why a 3-state cycle instead of only asc/desc?

It lets users get back to the original (e.g. "relevance" or "newest") order without a separate control. Some users find it confusing, which is why there's also a Reset button and a text label.

Q20. How would you keep the sort after a refresh?

Mirror `{ key, direction }` into the URL (`?sort=price&dir=desc`) with `history.replaceState`, read it on load with validation (unknown keys fall back to no sort), or store it in localStorage for a per-user preference.